

Perfect Square

279. Perfect Squares

Medium

Topics



Companies

Given an integer n , return *the least number of perfect square numbers that sum to n* .

A **perfect square** is an integer that is the square of an integer; in other words, it is the product of some integer with itself. For example, 1, 4, 9, and 16 are perfect squares while 3 and 11 are not.

Example 1:

Input: $n = 12$

Output: 3

Explanation: $12 = 4 + 4 + 4$.

Example 2:

Input: $n = 13$

Output: 2

Explanation: $13 = 4 + 9$.

understand from examples:

*return least no of
perfect square.*

ex:

n = 12

no of perfect square



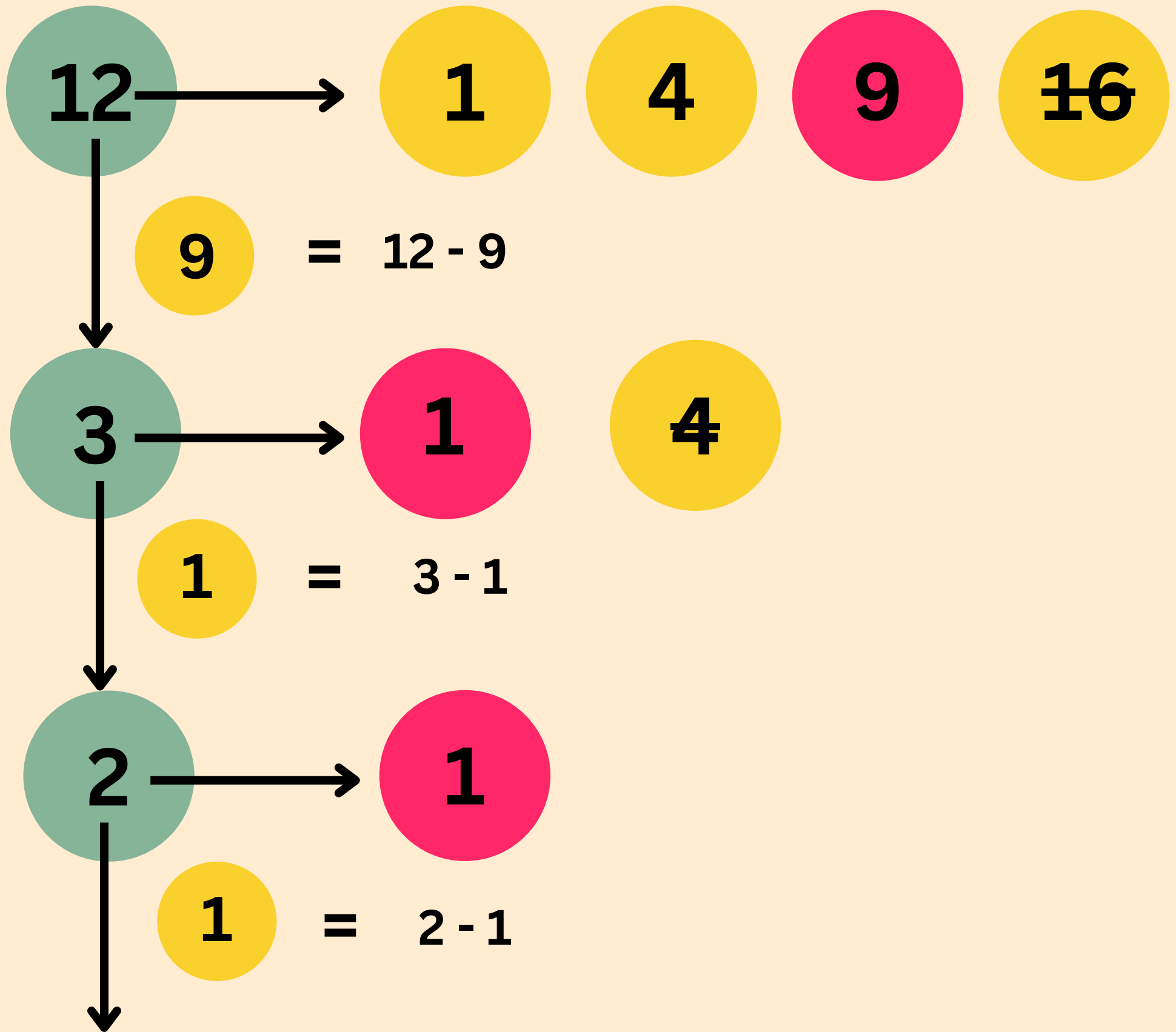
first ans $1+1+1+1+1+1+1+1+1+1+1+1 = 12$

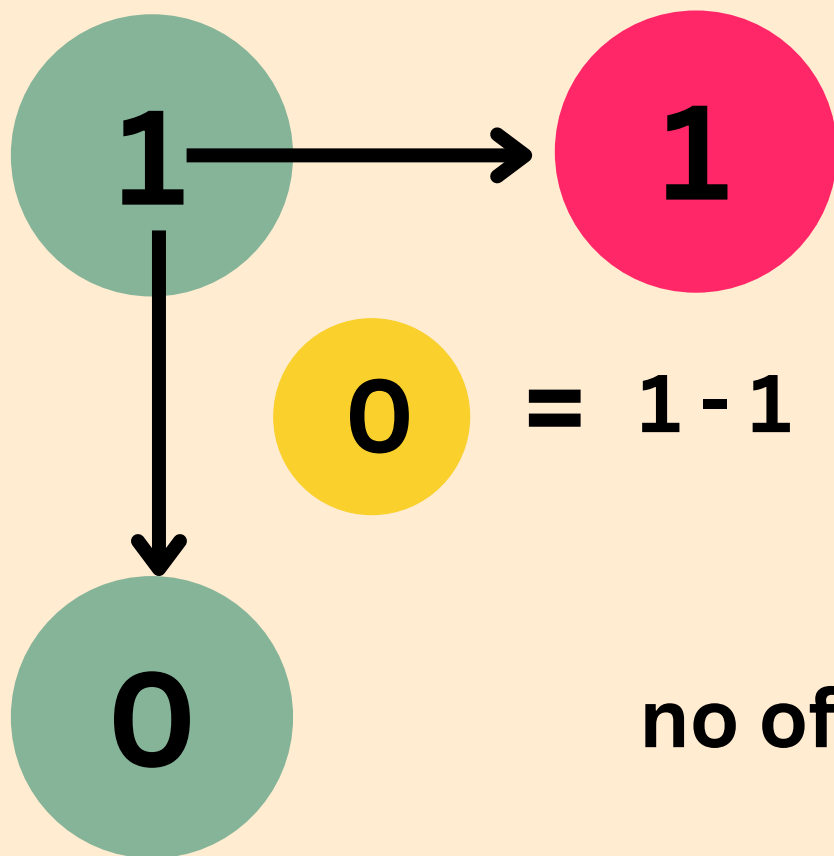
2nd ans: $4 + 4 + 4 = 12$ ← no of perfect square
is 3

output: 3 ← *minimum no of perfect
square to make 12*

Greedy approach:

- write perfect square less than equal to $n=12$





$$0 = 1 - 1$$

no of red circle is 4

$$9 + 1 + 1 + 1 = 12$$

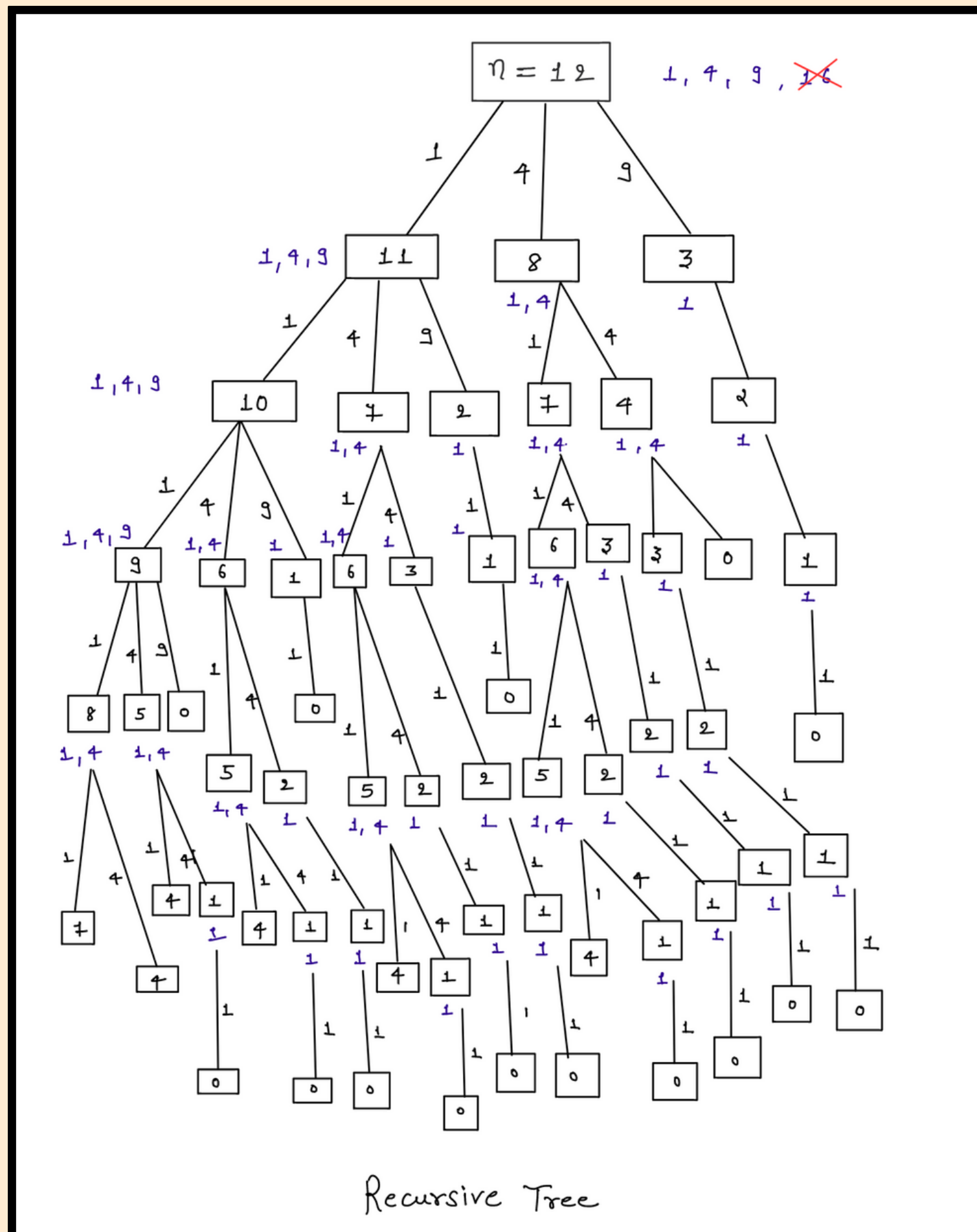
$$\text{ans} = 4$$

But

$$\text{Right ans} = 3$$

$$4 + 4 + 4$$

- here, greedy approach fail.



recursive code in c++:-

C++  Auto

```
1 class Solution {
2 public:
3     int fun(int ind, int tar, int n)
4     {
5         if(ind * ind > n){
6             if(!tar){
7                 return 0;
8             }
9             return 1e9;
10        }
11        else if(ind*ind == n){
12            return 1;
13        }
14        int take = 1e9;
15        if(tar >= ind*ind){
16            take = 1 + fun(ind, tar - ind*ind, n);
17        }
18        int not_take = fun(ind+1, tar,n);
19        return min(take, not_take);
20    }
21    int numSquares(int n) {
22        return fun(1, n, n);
23    }
24 };
```

Accepted Runtime: 0 ms

• Case 1 • Case 2

Input

n =

12

Output

3

Expected

3

Time Limit Exceeded

256 / 588 testcases passed

Last Executed Input

 Use Testcase

n =

255

@ manishkumar_978

$$T.c = O(2^n)$$

$$S.c = O(\sqrt{n})$$

Recursion + memoization:-

```
C++ v Auto
1 class Solution {
2 public:
3     int fun(int ind, int tar, int n, vector<vector<int>>& dp)
4     {
5         if(ind * ind > n){
6             if(!tar){
7                 return 0;
8             }
9             return 1e9;
10        }
11        else if(ind*ind == n){
12            return 1;
13        }
14        if(dp[ind][tar] != -1){
15            return dp[ind][tar];
16        }
17        int take = 1e9;
18        if(tar >= ind*ind){
19            take = 1 + fun(ind, tar - ind*ind, n, dp);
20        }
21        int not_take = fun(ind+1, tar, n, dp);
22        return dp[ind][tar] = min(take, not_take);
23    }
24    int numSquares(int n) {
25        vector<vector<int>> dp(sqrt(n)+1, vector<int>(n+1, -1));
26        return fun(1, n, n, dp);
27    }
28 };
```

Accepted

Manish Kumar submitted at Feb 09, 2024 00:26

Editorial

Solution

Runtime

428 ms

Beats 8.80% of users with C++

$T.c = O(\sqrt{n} * n)$

$S.c = O(\sqrt{n} * n)$

@ manishkumar_978



follow me on:



<https://www.linkedin.com/in/manish-kumar-689905213/>



manishkumar_978